

# Reproducing TableRAG on HybridQA and an Experimental Evaluation in Vietnamese

<Huynh Thanh Quan; Pham Ha Phuong; Nguyen Thi Thanh Xuan; Dang Thi Khanh Linh\_University of Science, Vietnam National University Ho Chi Minh City (HCMUS)>

## Abstract

Conventional RAG systems linearize tables into text, weakening row-column relationships and the ability to perform filtering or aggregation. This study presents an adapted replication of TableRAG, a method that combines vector retrieval with SQL execution over the original tables, on 305 HybridQA questions and 50 Vietnamese financial questions from VinumQA. Four API-based language models are compared with NaiveRAG in terms of accuracy, latency, and the number of reasoning iterations. On HybridQA, TableRAG outperforms NaiveRAG with all four models, improving accuracy by an average of 14.59 percentage points; Claude Haiku 4.5 reaches 71.80%, while Grok Build 0.1 records the largest gain. On VinumQA-50, average accuracy increases from 45% to 52%, but Claude decreases by 4 points and Grok remains unchanged. TableRAG latency is 3.0 to 15.6 times higher. The main errors occur in retrieval, schema selection, SQL generation, and answer normalization. The results reproduce the trend reported in the original study but do not constitute a direct quantitative replication because the models, configurations, and evaluation procedures differ.

## 1 Introduction

Retrieval-augmented generation (RAG) combines language models with external evidence (Lewis et al., 2020). When tables are converted to Markdown, HTML, or CSV and then segmented, headers may be separated from data cells and row-column relationships may be weakened. This representation is suitable for local lookup but less reliable for filtering, counting, sorting, joining, and aggregation over an entire table.

TableRAG addresses this limitation by retaining the original tables in a relational database and combining text retrieval with SQL

execution (Yu et al., 2025). Our study investigates two questions: whether the trend of TableRAG outperforming NaiveRAG can be reproduced when the backbone is changed, and how the method performs on Vietnamese financial data. Our contribution is an adapted replication on HybridQA, an experimental evaluation on VinumQA-50, and an analysis of the trade-offs among accuracy, latency, and errors at each stage.

## 2 Related Work

HybridQA requires multi-step reasoning over Wikipedia tables and linked text passages; if either source is missing, a question may become unanswerable (Chen et al., 2020). Text-only RAG can retrieve relevant cells but does not ensure a complete structure for operations over multiple rows. TableRAG simultaneously maintains a flattened table representation for retrieval and a relational table for SQL execution. BGE-M3 is used because it supports multilingual retrieval and inputs of varying lengths (Chen et al., 2024).

## 3 Data and Method

### 3.1 Datasets

**HybridQA.** The experiment uses the my\_dev.json development file and processes 305 English questions. VinumQA-50 is a subset selected by our group from a publicly released dataset on Kaggle, comprising 50 Vietnamese questions, 50 Excel tables, and 50 JSON documents. The data are primarily financial and cover lookup, filtering, comparison, and calculation; the questions were not translated into English.

### 3.2 TableRAG

Given unstructured text  $T$ , a set of tables  $D$ , and a question  $q$ , the system finds the answer:

$$\mathcal{A} = \mathcal{F}(\mathcal{D}, \mathcal{T}, q) \quad (1)$$

During the offline phase, text and Markdown tables are segmented; BAAI/bge-m3 encodes each segment into a 1,024-dimensional vector, and FAISS IndexFlatIP stores the vectors. Each table segment is mapped to its source schema, while the original table is loaded into MySQL or MariaDB.

$$\mathcal{C} = \text{Chunk}(\mathcal{T} \cup \hat{\mathcal{D}}), \quad v(c) = E(c) \in \mathbb{R}^{1024} \quad (2)$$

In each online iteration, the LLM: (1) decomposes the question; (2) retrieves and reranks 30 segments; (3) generates and executes SQL when tabular evidence is needed; and (4) integrates the SQL results with the retrieved text. The loop stops when sufficient evidence has been obtained or after five iterations.

$$e_{\square} = \text{fSQL}(S^t, q_{\square}), \quad \mathcal{A} = a_{\square}, \quad T \leq 5 \quad (3)$$

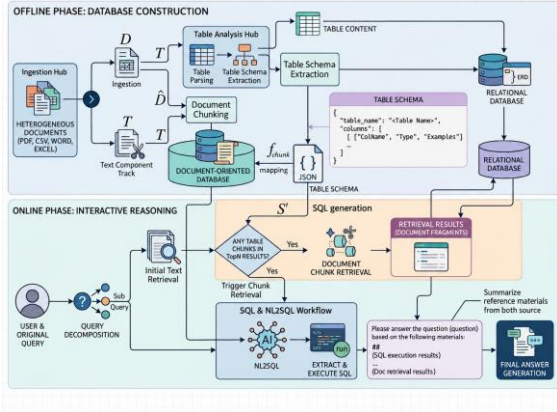


Figure 1: The two-stage architecture of TableRAG (Yu et al., 2025).

The mapping between table segments and schemas bridges the two stages: retrieval identifies the relevant table, while the relational database performs operations over the entire table. The SQL results and text segments after reranking are jointly passed to the answer synthesis step.

### 3.3 Replication Configuration

The experiments were run on Google Colab T4 with qwen-flash, deepseek-v4-flash, grok-build-0.1, and claude-haiku-4-5 at a temperature of 0.1. Text was segmented into 1,000-character chunks with an overlap of 200 characters; BGE-M3 input was limited to 256 tokens; embedding and reranking batch sizes were 32. The system retained five segments after reranking and fed the top three into the reasoning process. NaiveRAG used the same embedding model,

reranker, chunking strategy, and top-k, but generated the answer in a single pass, without SQL execution or iterative decomposition.

This configuration differs from the original paper in the chunking unit, input limit, and number of segments retained after reranking. Therefore, comparison with the authors' results is used only to assess the trend and is not treated as a direct quantitative comparison.

## 4 Results and Discussion

### 4.1 HybridQA

| Backbone | Naive | Table | $\Delta$ |
|----------|-------|-------|----------|
| Claude   | 52,13 | 71,80 | +19,67   |
| Qwen     | 24,92 | 34,75 | +9,84    |
| Grok     | 15,41 | 39,02 | +23,61   |
| DeepSeek | 37,70 | 42,95 | +5,25    |

Table 1: Accuracy (%) on HybridQA, n = 305.

TableRAG outperformed NaiveRAG across all four backbones, increasing the macro average from 32.54% to 47.13%, a gain of 14.59 percentage points. Claude achieved the highest accuracy; Grok improved the most, although its final result remained below Claude's. DeepSeek showed the smallest gain, indicating that adding SQL does not eliminate dependence on the backbone's capabilities in decomposition, SQL generation, and synthesis.

This trend is consistent with Yu et al. (2025), who reported HybridQA gains of 27.59 points with Claude-3.5, 21.31 with DeepSeek-V3, and 25.90 with Qwen-2.5-72B. Our gains were smaller for the three comparable model families. Differences in backbones and the reduced configuration used on Colab T4 are plausible explanations, but their effects cannot be isolated without an ablation study.

### 4.2 VinumQA-50

| Backbone | Naive | Table | $\Delta$ |
|----------|-------|-------|----------|
| Claude   | 76    | 72    | -4       |
| Grok     | 34    | 34    | 0        |
| Qwen     | 24    | 46    | +22      |
| DeepSeek | 46    | 56    | +10      |

Table 2: Accuracy (%) on VinumQA-50.

Average accuracy increased from 45% to 52%, corresponding to 14 additional correct answers across the four backbones. Qwen gained 22 points and DeepSeek gained 10 points; Grok remained unchanged, while Claude decreased by

4 points. Because the language, domain, and evaluation sample differ, the VinumQA values cannot be directly compared with the HybridQA results in the original paper. The results only show that the benefit of structured retrieval depends strongly on the backbone and question type.

#### 4.3 Latency and Number of Reasoning Iterations

| Dataset-model | N (s) | T (s) | T steps |
|---------------|-------|-------|---------|
| HQA-Claude    | 12,78 | 38,11 | 2,68    |
| HQA-Qwen      | 3,36  | 14,83 | 2,42    |
| HQA-Grok      | 12,32 | 67,25 | 4,20    |
| HQA-DeepSeek  | 6,31  | 85,56 | 2,96    |
| VQA-Claude    | 4,13  | 30,16 | 2,04    |
| VQA-Qwen      | 4,01  | 21,01 | 2,40    |
| VQA-Grok      | 21,31 | 84,18 | 4,08    |
| VQA-DeepSeek  | 5,32  | 82,98 | 1,78    |

Table 3: NaiveRAG latency (N), TableRAG latency (T), and average number of steps.

TableRAG was slower in every configuration, by approximately 3.0-15.6 times, because it invokes the LLM and performs retrieval over multiple iterations. Grok had the highest average number of iterations on both datasets. On VinumQA, 32 of 50 Grok questions reached the five-iteration limit, and 28 produced no answer. More iterations therefore often indicate failure to converge rather than higher accuracy.

#### 4.4 Interpretation of Results

On HybridQA, TableRAG produced 60 additional correct answers with Claude, 30 with Qwen, 72 with Grok, and 16 with DeepSeek. The gain across every backbone supports a fairly consistent qualitative conclusion: preserving table structure and executing SQL is more useful than feeding the LLM only three text segments. However, the final ranking still depends strongly on the backbone. NaiveRAG with Claude achieved 52.13%, higher than every other TableRAG configuration; the retrieval architecture does not fully compensate for differences in model capability.

On VinumQA-50, variation across backbones was more pronounced. TableRAG helped Qwen and DeepSeek but did not help Grok, while reducing Claude's result by two correct answers. Because each question corresponds to two percentage points, small differences are

insufficient to establish a stable advantage. The Claude case shows that direct lookup questions can be overprocessed: unnecessary decomposition, incorrect schema selection, or erroneous SQL can corrupt evidence that NaiveRAG retrieved correctly.

Time cost is also not proportional to accuracy. On HybridQA, latency increased by approximately 3.0 times with Claude, 4.4 times with Qwen, 5.5 times with Grok, and 13.6 times with DeepSeek. On VinumQA, the corresponding factors were approximately 7.3, 5.2, 4.0, and 15.6 times. Therefore, practical deployment requires complexity-based routing: one-step lookup questions should use NaiveRAG, whereas TableRAG should be activated only for questions involving filtering, extrema, aggregation, or multiple tables. Caching embeddings, retrieval results, and schemas is also necessary to reduce repeated API calls.

#### 5 Error Analysis

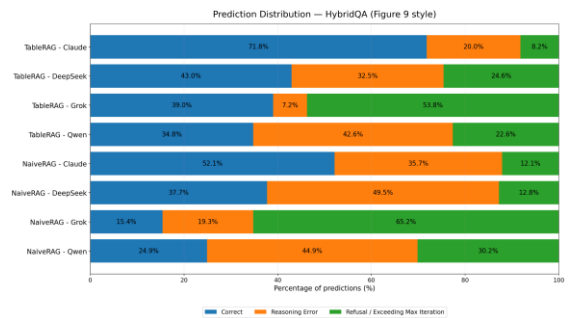


Figure 2: Distribution of TableRAG and NaiveRAG predictions on HybridQA.

In addition to evaluating TableRAG's overall performance against the existing baseline, we conducted a detailed error analysis to clarify the nature of failed predictions. Failed outputs were divided into two main groups: failures during reasoning, caused by SQL execution errors or inappropriate decomposition of intermediate queries; and failure to complete the task, typically manifested as refusal to answer or termination after reaching the maximum iteration limit.

The distribution of model prediction outcomes is shown in Figure 2. For most models, TableRAG reduces reasoning errors and the number of incomplete tasks compared with NaiveRAG. This suggests that adding query decomposition and structured reasoning over tables helps the system handle questions that require multiple reasoning steps.

Claude reduced both reasoning errors and incomplete cases. With DeepSeek, reasoning errors decreased, but the rate of refusals or iteration-limit failures increased. Grok still had 53.8% incomplete cases, indicating that the convergence bottleneck remained unresolved.

In the error traces for Qwen and DeepSeek, 78.2% of causes were associated with text/document retrieval, 13.5% with SQL, and 6.6% with final-answer generation; other categories accounted for small proportions. Typical errors included selecting the wrong table or column, using an incorrect aggregation, storing numbers as text, failing to match Vietnamese financial abbreviations, missing evidence segments, and altering a correct value during synthesis. Priority measures include metadata filtering, number and unit normalization, an abbreviation dictionary, SQL identifier validation, read-only permissions, and verification of the final answer against the execution result.

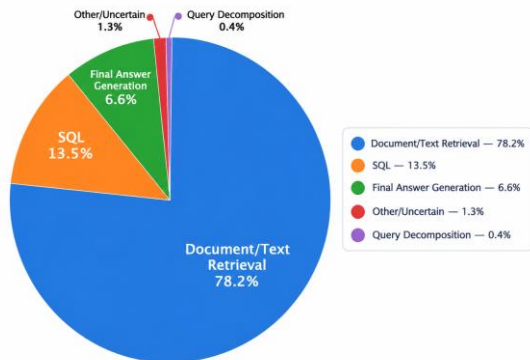


Figure 3: Distribution of reasoning-error causes in Qwen and DeepSeek traces.

The proportion of retrieval errors shows that SQL is not the only bottleneck. If the input segment or table is already incorrect, subsequent iterations only amplify the error. Therefore, improvements should prioritize hybrid keyword-vector retrieval and filtering by company and reporting period, followed by SQL correction using database error feedback.

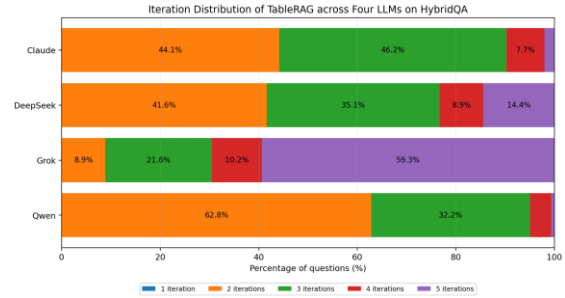


Figure 4: Distribution of TableRAG iteration counts on HybridQA.

Claude and Qwen mostly stopped at iterations 2-3, whereas 59.3% of Grok queries continued to iteration 5. This difference partly explains the latency differences among backbones and shows that the stopping criterion strongly depends on the LLM's ability to judge whether the evidence is sufficient.

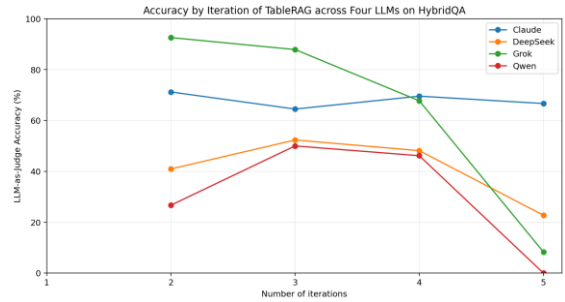


Figure 5: Accuracy by number of iterations on HybridQA.

Accuracy did not increase monotonically with the number of iterations. For Grok and Qwen, the group of queries that reached iteration 5 had very low accuracy; for DeepSeek, results also declined markedly in the final iteration. A large iteration count should therefore be treated as a signal of a difficult question or non-convergent reasoning rather than as a budget that should be expanded by default.

## Limitations

VinumQA-50 contains only 50 questions, so each question changes accuracy by 2 percentage points; the subset was not sampled using stratification and is primarily financial. The experiments did not include repeated runs, statistical tests, or ablation studies, and the results remain dependent on API versions. Differences in backbones, chunking, BGE limits, and the evaluation procedure mean that the results cannot be regarded as a direct quantitative replication of the original paper.

## Ethical Considerations

Financial data should be checked for usage rights and stripped of personal information. SQL generated by an LLM should run only with read permissions, access limits, and logging. The results should not be used directly for credit approval or fraud determinations; decisions that affect customers must be reviewed by specialists together with the evidence and SQL queries.

## 6 Conclusion

We conducted an adapted replication of TableRAG and reproduced the trend of the method outperforming NaiveRAG on HybridQA, although the gain was smaller than in the original paper. The VinumQA-50 experiment showed that performance in Vietnamese was inconsistent across backbones, while latency increased substantially. A suitable deployment strategy is to route simple lookup questions to NaiveRAG and use TableRAG only when table operations are required; future research should address Vietnamese Text-to-SQL, SQL-evidence verification, and larger evaluation sets.

## References

- Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. Findings of ACL 2024, 2318-2335. <https://doi.org/10.18653/v1/2024.findings-acl.137>
- Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. 2020. HybridQA: A Dataset of Multi-Hop Question Answering over Tabular and Textual Data. Findings of EMNLP 2020, 1026-1036. <https://doi.org/10.18653/v1/2020.findings-emnlp.91>
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems 33, 9459-9474.
- Xiaohan Yu, Pu Jian, and Chong Chen. 2025. TableRAG: A Retrieval Augmented Generation Framework for Heterogeneous Document Reasoning. EMNLP 2025; arXiv:2506.10380. <https://doi.org/10.48550/arXiv.2506.10380>
- dintrn. n.d. ViNumQA. Kaggle dataset. Accessed September 6, 2026. <https://www.kaggle.com/dintrn/vinumqa/data?select=train.json>